
MILESTONE 2

Structure

1. Data Quality Report
2. Data Preparation & Feature Engineering
3. Appendix (Code)

MILESTONE 2: DATA QUALITY AND PREPARATION REPORT

1. Executive Summary

The diabetes dataset was cleaned and prepared for modeling in two main steps. First, we performed a **Data Quality Assessment** (Step 4): all 22 features were checked for missing values and duplicate records. No missing entries were found. Approximately **23,899 duplicate records (9.42% of 253,680)** were identified and removed, reducing the dataset to **229,781 rows**.

Next, **Data Preparation and Feature Engineering** (Step 5) were applied.

The cleaned data had its index reset after removing duplicate records. One new feature was created: **BMI_cat**, which categorizes BMI into standard health categories (0 = Underweight, 1 = Normal, 2 = Overweight, 3 = Obese).

The original target variable **Diabetes_012** was retained in its original multi-class form (0, 1, 2) to preserve complete information for modeling. This allows the problem to be treated as a multi-class classification task.

The final dataset contains **229,781 rows and 22 columns**, and is ready for further analysis and modeling.

Code Implementation

```
import pandas as pd

# Load dataset
df = pd.read_csv(r"C:\Users\max18\OneDrive\Desktop\vlba\diabetes.csv",
sep=";")

# -----
# DATA QUALITY CHECK
# -----

# Missing values
print("\nMissing values per column:")
print(df.isnull().sum())

# Duplicate rows
dup_count = df.duplicated().sum()
dup_pct = dup_count / len(df) * 100
print(f"\nDuplicate rows found: {dup_count} ({dup_pct:.1f}%)")

# Show sample duplicates
print("\nSample duplicate rows (first 5):")
print(df[df.duplicated(keep='first')].head(5))

# -----
# DATA CLEANING
# -----

# Remove duplicates and reset index
df = df.drop_duplicates().reset_index(drop=True)

print("\nShape after removing duplicates:", df.shape)

# -----
# FEATURE ENGINEERING (ONLY BMI)
```

```

# -----

def bmi_category(bmi):
    if bmi < 18.5:
        return 0
    elif bmi < 25:
        return 1
    elif bmi < 30:
        return 2
    else:
        return 3

df['BMI_cat'] = df['BMI'].apply(bmi_category)

# -----
# FINAL CHECK
# -----

print("\nColumns:", df.columns)

# Save cleaned dataset
df.to_csv("diabetes_cleaned.csv", index=False)

print("\nCleaned dataset saved as 'diabetes_cleaned.csv'")

```

```
PS C:\Users\max18> & C:/Users/max18/AppData/Local/Microsoft/WindowsApps/python3.12.exe "c:/Users/max18/OneDrive/Desktop/vlba/import_pandas_as_pd.py"
```

Missing values per column:

```

Diabetes_012      0
HighBP            0
HighChol          0
CholCheck         0
BMI               0
Smoker            0
Stroke            0
HeartDiseaseorAttack 0
PhysActivity      0
Fruits            0
Veggies           0
HvyAlcoholConsump 0
AnyHealthcare     0
NoDocbcCost       0
GenHlth           0
MentHlth          0
PhysHlth          0
DiffWalk          0
Sex               0
Age               0
Education          0
Income            0
dtype: int64

```

Duplicate rows found: 23899 (9.4%)

Sample duplicate rows (first 5):

	Diabetes_012	HighBP	HighChol	CholCheck	BMI	Smoker	Stroke	HeartDiseaseorAttack	PhysActivity	...	NoDocbcCost	GenHlth	MentHlth	PhysHlth	DiffWalk	Sex	Age	Education	Income
1242	2.0	1.0	1.0	1.0	27.0	1.0	0.0	0.0	0.0	...	0.0	5.0	0.0	30.0	1.0	0.0	10.0	4.0	5.0
1563	0.0	0.0	0.0	1.0	21.0	1.0	0.0	0.0	1.0	...	0.0	1.0	0.0	0.0	0.0	0.0	4.0	6.0	8.0
2700	0.0	0.0	0.0	1.0	32.0	0.0	0.0	0.0	1.0	...	0.0	2.0	0.0	0.0	0.0	0.0	5.0	6.0	8.0
3160	0.0	0.0	0.0	1.0	21.0	0.0	0.0	0.0	1.0	...	0.0	1.0	0.0	0.0	0.0	0.0	4.0	6.0	8.0
3332	0.0	0.0	0.0	1.0	24.0	0.0	0.0	0.0	1.0	...	0.0	1.0	0.0	0.0	0.0	1.0	9.0	6.0	8.0

[5 rows x 22 columns]

Shape after removing duplicates: (229781, 22)

The output confirms that no missing values are present and 23,899 duplicate rows (9.42%) were identified and removed.

Distribution of Target Variable (Diabetes_012)

The target variable **Diabetes_012** contains three classes:

- 0 → No diabetes
- 1 → Prediabetes
- 2 → Diabetes

A histogram plot was used to analyze the distribution of the target variable.

Observation:

- Class 0 is significantly more frequent than the other classes
- Classes 1 and 2 are comparatively underrepresented

This indicates a **class imbalance problem** in the dataset.

Handling Class Imbalance

Class imbalance can negatively impact model performance by biasing predictions toward the majority class.

To address this issue, techniques such as **SMOTE (Synthetic Minority Oversampling Technique)** will be applied during the modeling phase to balance the class distribution.

Detailed Data Quality Report

Dataset Overview

The **diabetes.csv** dataset contains *health-related features* for patients, intended for a diabetes risk prediction model. Features include **BMI, age, blood pressure, cholesterol checks, smoking status, physical activity**, and more. The dataset has 22 columns, mixing numeric and categorical indicators. Before analysis, we assessed data quality (missing values, duplicates) to ensure a reliable training set.

Data Loading and Formatting

Originally, the data had a loading issue where all fields appeared in one column due to delimiter problems. The dataset was initially loaded incorrectly due to delimiter mismatch. This was resolved by specifying the correct separator (;) in pandas. After fixing the delimiter, the data was reloaded into Pandas with the correct structure for analysis.

Missing Values Analysis

We checked each column for missing or null entries using Pandas' `DataFrame.isnull()`. This function returns a boolean mask indicating where values are NA. In our dataset, **no missing values were detected in any column**. All columns had complete data (zero missing entries), indicating that imputation was not required.

"DataFrame.isnull()... returns a boolean same-sized object indicating if the values are NA".

Duplicate Records Analysis

We used Pandas' `DataFrame.duplicated()` to identify duplicate rows. This method returns a boolean Series marking duplicates (by default, keeping the first occurrence as non-duplicate). The analysis found **23,899 duplicate rows**, which is approximately **9.4%** of the original 253,680 rows. Duplicates were detected across all columns (no subset specified), meaning these rows were exact repeats of earlier entries.

Metric	Count	Percentage
Total rows (before)	253,680	–
Duplicate rows found	23,899	9.4%

Duplicates can bias machine learning models by **overrepresenting repeated cases and skewing patterns**. They may cause the model to learn spurious patterns that do not generalize. For robust modeling, duplicates should be removed to ensure each example contributes uniquely.

Handling of Duplicates

All duplicate rows were removed using `DataFrame.drop_duplicates()`. This returns a new DataFrame with duplicate entries dropped. We reset the DataFrame index after removal for cleanliness. After deduplication, the dataset shape decreased from 253,680 rows to 229,781 rows, with the same 22 columns.

Example transformation:

```
df_cleaned = df.drop_duplicates().reset_index(drop=True)
```

The removal step reduced the data volume by the identified percentage. A summary of the dataset shape is shown below.

1. Missing Values Check

We examined each column for missing values using pandas (`df.isnull().sum()`). **Result:** No missing values were found in any of the 22 columns. This indicates the dataset is complete and requires no imputation.

2. Duplicate Records

We checked for duplicate rows with `df.duplicated().sum()`. The analysis revealed:

- **Total duplicates:** 23,899 rows

- **Percentage of duplicates:** 9.42% of the original 253,680 rows

Duplicate records can bias the model by over-weighting repeated observations. To ensure data integrity, **all duplicate rows were removed** using `df.drop_duplicates()`.

Shape Before and After Cleaning

Dataset State	Rows	Columns
Before cleaning (original)	253,680	22
After removing duplicates	229,781	22

This table shows that 23,899 rows were removed, with the column count unchanged (22 features).

Outlier Detection

Outlier detection was conducted to identify extreme values in continuous features that could potentially affect model performance. The analysis focused on numerical variables such as **BMI**, **Age**, **MentHlth**, and **PhysHlth**, which are more likely to contain non-typical values.

The **Interquartile Range (IQR) method** was used to detect outliers. For each feature, the first quartile (Q1) and third quartile (Q3) were computed, and the IQR was defined as $IQR = Q3 - Q1$. Observations falling below $Q1 - 1.5 \times IQR$ or above $Q3 + 1.5 \times IQR$ were considered outliers.

The analysis revealed that:

- **BMI** exhibits a noticeable number of outliers, particularly in the upper range, indicating cases of extreme obesity.
- **MentHlth** and **PhysHlth** show skewed distributions with high-value outliers (close to the maximum of 30 days), reflecting prolonged periods of poor mental or physical health.
- **Age** demonstrates a relatively stable distribution with fewer extreme values.
- Binary variables (e.g., HighBP, Smoker) do not contain outliers due to their categorical nature.

These outliers were **not removed** from the dataset, as they represent realistic medical conditions rather than data errors. Removing them could lead to loss of valuable information, particularly in healthcare-related analysis where extreme values may carry significant meaning.

Overall, the dataset contains some extreme values in continuous features, but they are valid observations and will be retained for modeling. Any impact of these values will be handled during the modeling stage if necessary.

The high number of outliers in MentHlth and PhysHlth is due to their bounded and highly skewed nature (0–30 days), where larger values indicate severe health conditions rather than anomalies.

Code Implementation

```
import pandas as pd

# Load dataset
df = pd.read_csv(r"C:\Users\max18\OneDrive\Desktop\vlba\diabetes.csv",
sep=";")

# Select numerical columns for outlier detection
numerical_cols = ['BMI', 'Age', 'MentHlth', 'PhysHlth']

print("==== OUTLIER DETECTION RESULTS ====")

for col in numerical_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1

    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    outliers = df[(df[col] < lower_bound) | (df[col] > upper_bound)]

    print(f"\nFeature: {col}")
    print(f"Number of outliers: {len(outliers)}")
    print(f"Lower bound: {lower_bound:.2f}, Upper bound: {upper_bound:.2f}")
```

```

PS C:\Users\max18> & C:/Users/max18/AppData/Local/Microsoft/WindowsApps/python3.12.exe "
===== OUTLIER DETECTION RESULTS =====

Feature: BMI
Number of outliers: 9847
Lower bound: 13.50, Upper bound: 41.50

Feature: Age
Number of outliers: 0
Lower bound: 0.00, Upper bound: 16.00

Feature: MentHlth
Number of outliers: 36208
Lower bound: -3.00, Upper bound: 5.00

Feature: PhysHlth
Number of outliers: 40949
Lower bound: -4.50, Upper bound: 7.50
PS C:\Users\max18>

```

3. Conclusion

Removing duplicates prevents the model from learning redundant patterns, improving generalization. After cleaning, the dataset has **229,781 rows and 22 columns**. We now have a high-quality dataset with no missing values and no duplicates, ready for feature engineering.

(B) Data Preparation & Feature Engineering (Step 5)

1. Final Data Cleaning

Starting from the cleaned data (229,781 rows, 22 columns), we reset the index to ensure a continuous index after row removal (`df.reset_index(drop=True)`). No further cleaning (e.g. imputation) was needed since the dataset had no missing values.

2. Feature Engineering

This was implemented using a custom function applied to the BMI feature. Categorizing BMI makes it easier to interpret the effect of obesity level on diabetes risk.

BMI Category (BMI_cat):

The continuous BMI feature was transformed into categorical groups using standard medical thresholds:

0 = Underweight (BMI < 18.5)

1 = Normal (18.5 ≤ BMI < 25)

2 = Overweight (25 ≤ BMI < 30)

3 = Obese (BMI ≥ 30)

The target variable Diabetes_012 was retained in its original multi-class form (0, 1, 2) to preserve complete information and support multi-class classification.

Final Dataset Structure

The final dataset has 229,781 rows and 22 columns, including the newly created BMI_cat feature.

The columns are:

['HighBP', 'HighChol', 'CholCheck', 'BMI', 'Smoker', 'Stroke', 'HeartDiseaseorAttack',
'PhysActivity', 'Fruits', 'Veggies', 'HvyAlcoholConsump', 'AnyHealthcare', 'NoDocbcCost',
'GenHlth', 'MentHlth', 'PhysHlth', 'DiffWalk', 'Sex', 'Age', 'Education', 'Income',
'BMI_cat', 'Diabetes_012']

Example distribution of the new feature:

BMI_cat:

Underweight (0) = 3,053

Normal (1) = 58,927

Overweight (2) = 82,859

Obese (3) = 84,942

Justification

Each transformation was documented and justified:

Duplicate records were removed to prevent biased learning

BMI_cat was created to improve interpretability of obesity-related risk

The original target variable (Diabetes_012) was retained to preserve full class information

These transformations introduce domain-informed feature engineering, allowing the model to better capture non-linear relationships and improving interpretability while maintaining alignment with healthcare standards.

Code Implementation

```
import pandas as pd

df = pd.read_csv(r"C:\Users\max18\OneDrive\Desktop\vlba\diabetes.csv",
sep=";")

print("Shape before:", df.shape)

# -----
# REMOVE DUPLICATES
# -----
df = df.drop_duplicates()
df = df.reset_index(drop=True)

print("Shape after:", df.shape)

# -----
# BMI CATEGORY
# -----
def bmi_category(bmi):
    if bmi < 18.5:
        return 0
    elif bmi < 25:
        return 1
    elif bmi < 30:
        return 2
    else:
        return 3

df['BMI_cat'] = df['BMI'].apply(bmi_category)

# -----
# FINAL CHECK
# -----
print("\nColumns:", df.columns)

print("\nBMI_cat:")
print(df['BMI_cat'].value_counts())

# SAVE
df.to_csv("diabetes_cleaned.csv", index=False)
```

```

PS C:\Users\max18> & C:/Users/max18/AppData/Local/Microsoft/WindowsApps/python3.12.exe "c
Shape before: (253680, 22)
Shape after: (229781, 22)

Columns: Index(['Diabetes_012', 'HighBP', 'HighChol', 'CholCheck', 'BMI', 'Smoker',
               'Stroke', 'HeartDiseaseorAttack', 'PhysActivity', 'Fruits', 'Veggies',
               'HvyAlcoholConsump', 'AnyHealthcare', 'NoDocbcCost', 'GenHlth',
               'MentHlth', 'PhysHlth', 'DiffWalk', 'Sex', 'Age', 'Education', 'Income',
               'BMI_cat'],
              dtype='object')

BMI_cat:
BMI_cat
3      84942
2      82859
1      58927
0       3053
Name: count, dtype: int64

```

Overall, the dataset preparation process ensures high data quality, improves feature interpretability, and establishes a strong foundation for reliable and robust machine learning modeling.